Vežba: zadatak klasifikacije teksta (TF-IDF, SVM i SMOTE)

U fajlu “mobile_apps_reviews.csv”[1] su dati komentari korisnika mobilnih aplikacija sa Google Play prodavnice. Svaki red je jedan komentar na jednu aplikaciju. Fajl sadrži sledeće kolone (varijable):

- **id** - redni broj komentara
- **app** - identifikator (ime paketa) aplikacije na koju se komentar odnosi
- **version** - verzija aplikacije u trenutku pisanja komentara
- **rating** - ocena aplikacije od 1 do 5 koju je korisnik dao uz komentar
- **date** - vreme objavljivanja komentara (Unix timestamp u milisekundama)
- **device** - model uređaja korisnika (“?” ako model nije poznat)
- **user** - ime korisnika koji je ostavio komentar
- **review** - tekst komentara
- **category** - ručno označen tip komentara (BUG, FEATURE, PERFORMANCE, USABILITY, ENERGY, SECURITY, OTHER)

Cilj je napraviti model koji automatski prepoznaje tip komentara, kako bi tim koji razvija aplikaciju brže pronašao prijave grešaka i zahteve za nove funkcionalnosti. Koristiti ceo skup podataka (3000 komentara).

Koristeći opisane podatke iz navedenog fajla, potrebno je uraditi sledeće:

- Učitati podatke i ispitati ih (tipovi kolona, nedostajuće vrednosti, duplikati). Izbaciti komentare kod kojih nedostaje tekst, kao i komentare sa duplikatom teksta.
- Polazeći od kolone **category** napraviti novu varijablu **tip_komentara** sa četiri moguće vrednosti, primenjujući sledeće mapiranje:
  - BUG se mapira u “greska”,
  - FEATURE se mapira u “funkcionalnost”,
  - PERFORMANCE, USABILITY, ENERGY i SECURITY se mapiraju u “nefunkcionalni”,
  - OTHER se mapira u “ostalo”.

  Varijabla tip_komentara bi trebalo da posluži kao izlazna varijabla za klasifikaciju u ovom zadatku. Prikazati broj i udeo komentara u originalnim kategorijama i u novim klasama, i ukratko obrazložiti zašto ima smisla spojiti četiri kategorije u jednu. Prokomentarisati šta neizbalansiranost klasa znači za kasniju evaluaciju.
- Proceniti da li bi još neku od varijabli iz polaznog dataset-a trebalo primeniti za kreiranje modela. Za svaku kolonu osim review ukratko obrazložiti odluku (uključiti / ne uključiti). Za kolonu rating odluku potkrepiti grafikom raspodele po klasama (box plot) i tabelom pd.crosstab normalizovanom po redovima, a za kolone app, version i device brojem različitih vrednosti. Posebno razmotriti da li je korišćenje kolone rating opravdano, tj. da li bi ta vrednost bila poznata u trenutku kada model treba da klasifikuje novi komentar. Model mora da koristi bar jednu varijablu pored teksta.
- Transformisati kolonu review u numerički oblik primenom Vector Space Modela i konkretno TF-IDF metrike, sa unigramima i bigramima. Pre računanja TF-IDF vrednosti tekst očistiti uz pomoć spaCy biblioteke (lematizacija, uklanjanje interpunkcije, brojeva,

URL-ova i stop reči, pri čemu reči “not” i “no” treba zadržati). Izabranu numeričku kolonu skalirati na opseg [0, 1] (MinMaxScaler) i u komentaru objasniti zašto je skaliranje atributa posebno važno kod SVM-a. Sve transformacije objediniti u jedan ColumnTransformer.

- Podeliti podatke na trening i test skup u odnosu 80:20, tako da udeo klasa bude isti u oba skupa. Kreirati prvi klasifikacioni model primenom algoritma Support Vector Machines (klasa SVC), sa podrazumevanim vrednostima hiperparametara. Transformacije i model objediniti u jedan Pipeline. U komentaru navesti koji kernel se podrazumevano koristi i šta predstavlja hiperparametar C.
- Napraviti predikcije primenom kreiranog modela nad test skupom i izračunati standardne evaluacione metrike (preciznost, odziv, F1) kako za svaku klasu pojedinačno, tako i sveukupno tj na nivou svih klasa. Prikazati matricu konfuzije. Interpretirati dobijene vrednosti metrika u kontekstu zadatka: koja klasa je najslabija, sa kojom klasom se najčešće meša (na osnovu matrice konfuzije) i šta to praktično znači za tim koji razvija aplikaciju. Posebno protumačiti situaciju kada klasa ima visoku preciznost, a nizak odziv.
- Kreirati drugi klasifikacioni model kroz postupak podešavanja hiperparametara. U komentaru navesti koje biste hiperparametre podešavali i na koji način tj koristeći koje opsege / liste vrednosti ako ne biste bili vremenski ograničeni (i za TF-IDF i za SVC: C, kernel, gamma, class_weight). Zatim, izabrati dva hiperparametra, jedan za transformaciju teksta (TF-IDF), drugi za klasifikator (C), i izvršiti podešavanje njihovih vrednosti kroz postupak (5-fold) kros-validacije, koristeći macro F1 kao metriku. Za svaki hiperparametar isprobati 2 do 3 vrednosti; vrednosti za C birati na logaritamskoj skali.
- Izvršiti predikcije primenom podešenog modela i izračunati standardne evaluacione metrike (preciznost, odziv, F1) kako za svaku klasu pojedinačno, tako i sveukupno tj na nivou svih klasa.
- Kreirati treći model koji pokušava da reši problem neizbalansiranosti klasa: u Pipeline podešenog modela, između ColumnTransformer-a i klasifikatora, dodati korak SMOTE (oversampling manjinskih klasa). Ostale hiperparametre zadržati sa vrednostima dobijenim u prethodnom koraku. Ukratko objasniti kako SMOTE kreira nove primere i zašto se primenjuje samo nad trening podacima, a ne pre podele na trening i test skup. U komentaru navesti i drugi način da se SVM-u ukaže na neizbalansiranost klasa, bez kreiranja novih primera.
- Uporediti i prokomentarisati rezultate dobijene osnovnim, podešenim i modelom sa SMOTE-om, koristeći datu funkciju get_model_metrics (strana 3). Kao donju granicu u poređenje uključiti i model koji uvek predviđa najčešću klasu (DummyClassifier). Da li je podešavanje poboljšalo model? Da li je SMOTE povećao odziv za najređe klase, i po koju cenu (preciznost)? Koji od modela biste preporučili timu i zašto?

[1] Podaci su preuzeti iz materijala sa vežbi (folder tm/data); fajl kopirati u folder data pored sveske.

Dati kod

Sledeću funkciju dobijate uz zadatak i možete je koristiti bez izmena:

```
def get_model_metrics(y_true: np.ndarray, y_pred: np.ndarray, model_name: str):
    report = metrics.classification_report(y_true, y_pred, output_dict=True)
    macro_avg = report['macro avg']
    weighted_avg = report['weighted avg']
    return {
        'Model': model_name,
        'Macro Precision': macro_avg['precision'],
        'Macro Recall': macro_avg['recall'],
        'Macro F1-Score': macro_avg['f1-score'],
        'Weighted Precision': weighted_avg['precision'],
        'Weighted Recall': weighted_avg['recall'],
        'Weighted F1-Score': weighted_avg['f1-score'],
        'Balanced Accuracy': metrics.balanced_accuracy_score(y_true, y_pred)
    }
```

Primer koda za box plot raspodele numeričke kolone po klasama:

```
plt.figure(figsize=(8, 6))
sb.boxplot(data=data, x='tip_komentara', y='rating', hue='tip_komentara',
           fill=True, saturation=0.75, width=0.55, gap=0.25,
           linewidth=2, legend=False)
plt.title('Raspodela ocena po tipu komentara')
plt.ylabel('Ocena')
plt.xlabel('')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

Importi potrebni za SMOTE:

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline   # zamenjuje sklearn.pipeline.Pipeline
```

Napomene za izradu

- Rešenje predati kao Jupyter svesku (.ipynb). Svaki korak zadatka započeti markdown ćelijom sa kratkim opisom, a tumačenja i obrazloženja pisati u markdown ćelijama, ne samo u komentarima koda.
- Svuda gde postoji parametar random_state (podela podataka, SMOTE, DummyClassifier) koristiti vrednost 1.
- SVC sa podrazumevanim kernelom radi direktno nad retkom (sparse) TF-IDF matricom; matricu ne pretvarati u gustu (toarray()), jer to troši mnogo memorije bez ikakve koristi.
- Pipeline iz sklearn biblioteke ne dozvoljava korak koji menja broj redova (SMOTE), pa se za treći model mora koristiti Pipeline iz imblearn biblioteke. Ako se oba importuju, voditi računa da jedno ime ne prepiše drugo (npr. from imblearn.pipeline import Pipeline as ImbPipeline).
- Kada ColumnTransformer sadrži više transformacija, ulaz u Pipeline mora biti DataFrame sa svim izabranim kolonama, a ne samo jedna kolona teksta.
- Ime hiperparametra u mreži za podešavanje sastoji se od imena koraka razdvojenih sa dve donje crte (korak u Pipeline-u, pa korak u ColumnTransformer-u, pa ime hiperparametra).

- Podešavanje hiperparametara raditi isključivo nad trening skupom; test skup koristiti samo za konačnu ocenu.
- Zbog spaCy čišćenja teksta u svakom foldu, podešavanje sa 6 kombinacija i 5 foldova traje nekoliko minuta. Ne povećavati mrežu preko 9 kombinacija.
- Brojevi sami po sebi nisu dovoljni: svaka metrika treba da bude protumačena u kontekstu komentara korisnika i tima koji razvija aplikaciju.